PixelPush

Stripe Integration Guide

Complete implementation reference for payments, subscriptions, webhooks & billing

---

Technical Documentation

Version 1.0

Developed by

Aepostrophee

Confidential · 2025

Table of Contents

Table of Contents 2
1. Overview 4
2. How Stripe Works — Core Concepts 4
  2.1 Customer 4
  2.2 Product & Price 4
  2.3 Checkout Session 5
  2.4 Webhook 5
  2.5 Customer Portal 5
3. Stripe Account Setup 6
  3.1 Create Your Account 6
  3.2 Create Your Product and Prices 6
  3.3 Configure the Customer Portal 6
4. Installation & Configuration 7
  4.1 Install Stripe SDK 7
  4.2 Environment Variables 7
  4.3 Stripe Utility Instance 7
5. Database Schema Changes 8
6. Checkout Flow Implementation 9
  6.1 Architecture 9
  6.2 Checkout API Route 9
  6.3 Frontend Upgrade Button 10
7. Webhook Implementation 12
  7.1 Register Your Webhook Endpoint 12
  7.2 Webhook Events Reference 12
  7.3 Webhook Route Implementation 12
8. Customer Portal — Self-Service Billing 16
  8.1 Portal API Route 16
  8.2 Manage Billing Button 16
9. Gating Pro Features 18
  9.1 Server-Side API Route Guard 18
  9.2 React Component Gate 18
  9.3 Free Tier Limits 19
10. Testing 20

10.1 Test Card Numbers 20
10.2 Testing Webhooks Locally 20
Install the Stripe CLI 20
Authenticate and Start Forwarding 20
Trigger Test Events Manually 21
10.3 Testing Checklist 21
11. Error Handling 22
11.1 Stripe API Errors 22
11.2 Idempotent Webhook Handling 22
12. Going Live Checklist 23
12.1 Stripe Dashboard 23
12.2 Environment Variables 23
12.3 Final Verification 23
13. Common Mistakes to Avoid 24
14. Stripe Fees & Revenue Impact 25
14.1 Stripe Transaction Fees 25
14.2 Net Revenue Per Plan 25
14.3 Break-Even Analysis 25
15. Glossary 26

1. Overview

This document is the complete Stripe integration reference for PixelPush. It covers everything required to implement a production-grade payment system: account setup, environment configuration, subscription and one-time payment flows, webhook handling, database schema, Pro feature gating, customer portal, testing, error handling, and going live.

PixelPush uses Stripe to handle two billing models simultaneously:

- Monthly recurring subscription at $7/month for PixelPush Pro
- One-time lifetime payment at $35 for permanent Pro access

Before You Begin

This guide assumes PixelPush is built on Next.js 14 with the App Router, uses Prisma with PostgreSQL (Neon), and authenticates users via NextAuth.js with GitHub OAuth. All code examples are written for this stack.

2. How Stripe Works — Core Concepts

Before writing a single line of code, understanding these five concepts is essential. Every Stripe integration — regardless of product — is built on top of them.

2.1 Customer

A Stripe Customer is a persistent object representing one of your users inside Stripe's system. It stores their payment methods, billing history, and active subscriptions. When a PixelPush user connects their GitHub account, you create a corresponding Stripe Customer and store the Customer ID in your database. Every future Stripe action — charging, subscribing, refunding — is performed against this Customer object.

Key Point

One PixelPush user = One Stripe Customer. Never create multiple Customers for the same user, and never charge a user without first retrieving or creating their Customer object.

2.2 Product & Price

A Product in Stripe represents what you are selling — in this case, PixelPush Pro. A Price defines how you are selling it: the amount, currency, and billing frequency. One Product can have multiple Prices attached to it.

Object	PixelPush Configuration
Product	PixelPush Pro — defined once in the Stripe Dashboard
Price (Monthly)	$7.00 USD · recurring · billed every month
Price (Lifetime)	$35.00 USD · one-time payment · no renewal

2.3 Checkout Session

When a user clicks Upgrade to Pro, your server creates a Checkout Session. Stripe returns a URL pointing to a hosted, Stripe-managed payment page. You redirect the user to that URL. Stripe handles the entire payment experience: card input, validation, 3D Secure authentication, error messaging, and receipts. Your server never touches raw card data.

Why Hosted Checkout?

Stripe's hosted Checkout page means you never handle raw card numbers. This keeps you out of PCI DSS scope — the strict security standard for card data — which would otherwise require extensive audits and compliance work.

2.4 Webhook

A webhook is an HTTP POST request that Stripe sends to your server when something happens — a payment succeeds, a subscription renews, a payment fails, or a user cancels. Your server listens at a dedicated webhook endpoint and reacts accordingly.

Critical Misunderstanding

Many beginners unlock Pro features when the user lands on the success URL after checkout. This is wrong and insecure. Anyone can visit that URL manually. The webhook is the only authoritative signal that a payment has actually been processed. Always gate Pro access on webhook events, never on URL redirects.

2.5 Customer Portal

Stripe provides a hosted Customer Portal — a pre-built UI where subscribers can manage their own billing: update payment methods, view invoices, and cancel subscriptions. You link users to this portal with a single API call. This means you never need to build a billing management interface yourself.

3. Stripe Account Setup

3.1 Create Your Account

1. Go to stripe.com and create a free account
2. Complete email verification
3. You will land in Test Mode by default — stay here until you are ready to go live
4. Navigate to Developers → API Keys in the left sidebar
5. Copy your Publishable Key (starts with pk_test_) and Secret Key (starts with sk_test_)

> **Secret Key Security**
> Your Secret Key must never be exposed in client-side code, committed to Git, or shared publicly. It grants full access to your Stripe account. Store it only in server-side environment variables.

3.2 Create Your Product and Prices

6. In the Stripe Dashboard, go to Product Catalog → Add Product
7. Name it PixelPush Pro, add a description, and save
8. Click Add Price on the product page
9. For the monthly plan: set $7.00, Recurring, Monthly — save and copy the Price ID (price_xxx)
10. Click Add Price again for the lifetime plan: set $35.00, One Time — save and copy this Price ID too
11. Store both Price IDs — you will reference them in your environment variables

3.3 Configure the Customer Portal

12. Go to Settings → Billing → Customer Portal in the Stripe Dashboard
13. Enable the portal and configure: allow cancellations, allow payment method updates, show invoice history
14. Save the settings — the portal will be accessible via an API-generated link

4. Installation & Configuration

4.1 Install Stripe SDK

```
npm install stripe @stripe/stripe-js
```

The stripe package is the Node.js server-side SDK. The @stripe/stripe-js package is only needed if you use Stripe Elements for custom card UIs — for PixelPush, which uses Stripe's hosted Checkout, it is optional but useful to have.

4.2 Environment Variables

Add the following to your .env.local file. Never commit this file to version control.

```
# Stripe API Keys
STRIPE_SECRET_KEY=sk_test_xxxxxxxxxxxxxxxxxxxx
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_xxxxxxxxxxxxxxxxxxxx

# Webhook signing secret (obtained in Step 5.1)
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxxxxxxxx

# Price IDs from your Stripe Dashboard
STRIPE_MONTHLY_PRICE_ID=price_xxxxxxxxxxxxxxxxxxxx
STRIPE_LIFETIME_PRICE_ID=price_xxxxxxxxxxxxxxxxxxxx

# Your app's public URL (used for redirect URLs)
NEXT_PUBLIC_URL=https://pixelpush.aepostrophee.com
```

4.3 Stripe Utility Instance

Create a single shared Stripe instance. Instantiating Stripe multiple times across your codebase is wasteful and can cause subtle bugs.

```
// lib/stripe.js
import Stripe from 'stripe';

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY, {
  apiVersion: '2024-06-20',
  typescript: true,
});
```

5. Database Schema Changes

Add the following fields to your User model in schema.prisma. These store the relationship between your users and their Stripe billing objects.

```
model User {
  // ... existing fields (id, github_id, username, etc.) ...

  // Stripe billing fields
  stripeCustomerId      String?   @unique
  stripeSubscriptionId  String?   @unique
  isPro                 Boolean   @default(false)
  proType               String?   // 'monthly' | 'lifetime'
  proSince              DateTime?
  proExpiresAt          DateTime? // null for lifetime, set for monthly
}
```

After editing the schema, run the Prisma migration:

```
npx prisma migrate dev --name add_stripe_fields
```

Field	Description
stripeCustomerId	Stripe's Customer ID (cus_xxx). Links your user to their Stripe billing record. Created on first checkout attempt.
stripeSubscriptionId	Stripe's Subscription ID (sub_xxx). Only populated for monthly subscribers. Null for lifetime buyers.
isPro	Boolean flag checked throughout the app to gate Pro features. Set to true on successful payment, false on cancellation or failed renewal.
proType	Either 'monthly' or 'lifetime'. Determines how isPro is managed — lifetime users never have their Pro access revoked.
proSince	Timestamp of when Pro was first activated. Useful for analytics and customer support.
proExpiresAt	For monthly subscribers, the end of the current billing period. If payment fails, Pro is revoked after this date.

6. Checkout Flow Implementation

6.1 Architecture

The checkout flow has three parts: a frontend button that calls your API, a server-side API route that creates the Checkout Session, and Stripe's hosted payment page that handles everything in between.

```
User clicks Upgrade
       |
       v
POST /api/stripe/checkout  <-- your Next.js API route
       |
       v  (creates session, returns URL)
Stripe Hosted Checkout Page  <-- user enters card details here
       |
       v  (payment processed)
Redirect to /dashboard?upgraded=true
       |
       v  (simultaneously, in background)
POST /api/stripe/webhook  <-- Stripe notifies your server
       |
       v
Database updated: isPro = true
```

6.2 Checkout API Route

This route runs entirely server-side. It creates or retrieves the Stripe Customer for the current user, then creates a Checkout Session for the selected plan.

```
// app/api/stripe/checkout/route.js
import { stripe } from '@/lib/stripe';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { db } from '@/lib/db';

export async function POST(req) {
  const session = await getServerSession(authOptions);
  if (!session) {
    return Response.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const user = await db.user.findUnique({
    where: { email: session.user.email }
  });

  if (user.isPro && user.proType === 'lifetime') {
    return Response.json({ error: 'Already a lifetime member' }, { status: 400 });
  }

  const { plan } = await req.json(); // 'monthly' or 'lifetime'
```

```
  // Create Stripe Customer if this user does not have one yet
  let stripeCustomerId = user.stripeCustomerId;
  if (!stripeCustomerId) {
    const customer = await stripe.customers.create({
      email: user.email,
      name: user.username,
      metadata: { userId: user.id }
    });
    stripeCustomerId = customer.id;
    await db.user.update({
      where: { id: user.id },
      data: { stripeCustomerId }
    });
  }

  const priceId = plan === 'lifetime'
    ? process.env.STRIPE_LIFETIME_PRICE_ID
    : process.env.STRIPE_MONTHLY_PRICE_ID;

  const checkoutSession = await stripe.checkout.sessions.create({
    customer: stripeCustomerId,
    payment_method_types: ['card'],
    line_items: [{ price: priceId, quantity: 1 }],
    mode: plan === 'lifetime' ? 'payment' : 'subscription',
    success_url: `${process.env.NEXT_PUBLIC_URL}/dashboard?upgraded=true`,
    cancel_url: `${process.env.NEXT_PUBLIC_URL}/pricing`,
    allow_promotion_codes: true,
    metadata: { userId: user.id, plan },
  });

  return Response.json({ url: checkoutSession.url });
}
```

6.3 Frontend Upgrade Button

The upgrade button on your pricing page calls the checkout route and redirects the user to Stripe's hosted page.

```
// components/UpgradeButton.jsx
'use client';
import { useState } from 'react';

export default function UpgradeButton({ plan }) {
  const [loading, setLoading] = useState(false);

  const handleUpgrade = async () => {
    setLoading(true);
    try {
      const res = await fetch('/api/stripe/checkout', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ plan }),
      });
```

```
      const { url, error } = await res.json();
      if (error) throw new Error(error);
      window.location.href = url;
    } catch (err) {
      console.error('Checkout error:', err);
      setLoading(false);
    }
  };

  return (
    <button onClick={handleUpgrade} disabled={loading}>
      {loading ? 'Redirecting...' : plan === 'lifetime' ? 'Get Lifetime — $35' :
'Upgrade to Pro — $7/mo'}
    </button>
  );
}
```

7. Webhook Implementation

The webhook is the backbone of your entire billing system. Every Pro access grant or revocation happens here — not in the checkout flow, not on the success page. This endpoint must be implemented correctly.

7.1 Register Your Webhook Endpoint

15. In the Stripe Dashboard, go to Developers → Webhooks
16. Click Add Endpoint
17. Set the URL to https://pixelpush.aepostrophee.com/api/stripe/webhook
18. Select the following events to listen for:

- checkout.session.completed
- invoice.paid
- invoice.payment_failed
- customer.subscription.updated
- customer.subscription.deleted

19. Save the endpoint and copy the Signing Secret (whsec_xxx) into your STRIPE_WEBHOOK_SECRET environment variable

7.2 Webhook Events Reference

Event	When It Fires & What To Do
checkout.session.completed	Fires when a user completes a checkout. Handles both monthly subscriptions and lifetime one-time payments. This is where Pro is first activated.
invoice.paid	Fires every time a monthly subscription invoice is paid successfully — i.e. every renewal. Keeps the user's isPro status active.
invoice.payment_failed	Fires when a renewal payment fails (expired card, insufficient funds). Revokes Pro access.
customer.subscription.updated	Fires when a subscription is modified — upgraded, downgraded, or scheduled for cancellation. Useful for future plan tiers.
customer.subscription.deleted	Fires when a subscription is cancelled and the billing period ends. Permanently revokes Pro for monthly subscribers.

7.3 Webhook Route Implementation

```
// app/api/stripe/webhook/route.js
import { stripe } from '@/lib/stripe';
import { db } from '@/lib/db';

export async function POST(req) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature');

  // Step 1: Verify the request is genuinely from Stripe
  let event;
  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET
    );
  } catch (err) {
    console.error('Webhook signature verification failed:', err.message);
    return new Response(`Webhook error: ${err.message}`, { status: 400 });
  }

  // Step 2: Handle each event type
  try {
    switch (event.type) {

      // Payment completed — grant Pro access
      case 'checkout.session.completed': {
        const session = event.data.object;
        const { userId, plan } = session.metadata;

        if (plan === 'lifetime') {
          await db.user.update({
            where: { id: userId },
            data: {
              isPro: true,
              proType: 'lifetime',
              proSince: new Date(),
              proExpiresAt: null,
            }
          });
        } else {
          // Fetch subscription to get period end date
          const subscription = await stripe.subscriptions.retrieve(
            session.subscription
          );
          await db.user.update({
            where: { id: userId },
            data: {
              isPro: true,
              proType: 'monthly',
              stripeSubscriptionId: session.subscription,
              proSince: new Date(),
              proExpiresAt: new Date(
                subscription.current_period_end * 1000
              ),
            }
          });
        }
        break;
      }
```

```
      // Monthly renewal succeeded — keep Pro active
      case 'invoice.paid': {
        const invoice = event.data.object;
        if (invoice.subscription) {
          const subscription = await stripe.subscriptions.retrieve(
            invoice.subscription
          );
          await db.user.update({
            where: { stripeSubscriptionId: invoice.subscription },
            data: {
              isPro: true,
              proExpiresAt: new Date(
                subscription.current_period_end * 1000
              ),
            }
          });
        }
        break;
      }

      // Payment failed — revoke Pro
      case 'invoice.payment_failed': {
        const invoice = event.data.object;
        if (invoice.subscription) {
          await db.user.update({
            where: { stripeSubscriptionId: invoice.subscription },
            data: { isPro: false }
          });
          // TODO: send email notifying user their payment failed
        }
        break;
      }

      // Subscription cancelled — revoke Pro at period end
      case 'customer.subscription.deleted': {
        const subscription = event.data.object;
        await db.user.update({
          where: { stripeSubscriptionId: subscription.id },
          data: {
            isPro: false,
            stripeSubscriptionId: null,
            proExpiresAt: null,
          }
        });
        break;
      }

      default:
        console.log(`Unhandled event type: ${event.type}`);
    }
  } catch (err) {
    console.error('Webhook handler error:', err);
    return new Response('Internal error', { status: 500 });
  }

  return new Response('OK', { status: 200 });
}
```

```
// Disable body parsing — Stripe needs the raw request body for signature
verification
export const config = { api: { bodyParser: false } };
```

8. Customer Portal — Self-Service Billing

The Customer Portal lets subscribers manage their own billing without you building any UI for it. They can update payment methods, download invoices, and cancel their subscription. You provide a link; Stripe handles everything else.

8.1 Portal API Route

```
// app/api/stripe/portal/route.js
import { stripe } from '@/lib/stripe';
import { getServerSession } from 'next-auth';
import { authOptions } from '@/lib/auth';
import { db } from '@/lib/db';

export async function POST(req) {
  const session = await getServerSession(authOptions);
  if (!session) {
    return Response.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const user = await db.user.findUnique({
    where: { email: session.user.email }
  });

  if (!user.stripeCustomerId) {
    return Response.json({ error: 'No billing account found' }, { status: 400 });
  }

  const portalSession = await stripe.billingPortal.sessions.create({
    customer: user.stripeCustomerId,
    return_url: `${process.env.NEXT_PUBLIC_URL}/dashboard`,
  });

  return Response.json({ url: portalSession.url });
}
```

8.2 Manage Billing Button

```
// components/ManageBillingButton.jsx
'use client';
import { useState } from 'react';

export default function ManageBillingButton() {
  const [loading, setLoading] = useState(false);

  const handlePortal = async () => {
    setLoading(true);
    const res = await fetch('/api/stripe/portal', { method: 'POST' });
    const { url } = await res.json();
    window.location.href = url;
```

```
  };

  return (
    <button onClick={handlePortal} disabled={loading}>
      {loading ? 'Loading...' : 'Manage Billing'}
    </button>
  );
}
```

9. Gating Pro Features

With isPro stored in your database, gating any Pro feature is a single database field check. Here are patterns for all the places you will need to enforce it.

9.1 Server-Side API Route Guard

Use this pattern in any API route that powers a Pro-only feature — AI project suggestions, unlimited theme generation, schedule recalculation, etc.

```
// Example: app/api/projects/suggest/route.js
import { getServerSession } from 'next-auth';
import { db } from '@/lib/db';

export async function POST(req) {
  const session = await getServerSession(authOptions);
  const user = await db.user.findUnique({
    where: { email: session.user.email }
  });

  // Pro gate
  if (!user.isPro) {
    return Response.json(
      { error: 'PixelPush Pro required for AI project suggestions' },
      { status: 403 }
    );
  }

  // ... proceed with AI suggestion logic
}
```

9.2 React Component Gate

Wrap Pro-only UI elements in a simple guard component that shows an upgrade prompt instead of the feature.

```
// components/ProGate.jsx
export default function ProGate({ isPro, children }) {
  if (isPro) return children;

  return (
    <div className='pro-gate'>
      <p>This feature is available on PixelPush Pro</p>
      <UpgradeButton plan='monthly' />
    </div>
  );
}

// Usage in any page:
```

```
// <ProGate isPro={user.isPro}>
//   <AIProjectSuggestions />
// </ProGate>
```

9.3 Free Tier Limits

Some features are available on both tiers but limited on free. Track usage counts in your database and enforce limits server-side.

```
// Enforce max 5 image suggestions for free users, 8 for Pro
const MAX_SUGGESTIONS = user.isPro ? 8 : 5;

// Enforce max 1 active design for free users, 3 for Pro
const activeDesigns = await db.design.count({
  where: { userId: user.id, status: 'active' }
});
const MAX_ACTIVE = user.isPro ? 3 : 1;

if (activeDesigns >= MAX_ACTIVE) {
  return Response.json(
    { error: `Free plan limited to ${MAX_ACTIVE} active design` },
    { status: 403 }
  );
}
```

10. Testing

10.1 Test Card Numbers

Stripe provides test card numbers that simulate specific payment outcomes. Use these while in Test Mode — they will never charge a real card.

Card Number	Outcome	Notes
4242 4242 4242 4242	Payment succeeds	Use any future expiry, any CVC
4000 0000 0000 0002	Payment declined	Generic decline
4000 0025 0000 3155	Requires 3D Secure	Tests authentication flow
4000 0000 0000 9995	Insufficient funds	Tests failed payment path
4000 0000 0000 0069	Expired card	Tests card expiry handling
4000 0000 0000 0341	Attaching fails	Card can be attached but charges fail

10.2 Testing Webhooks Locally

During development, Stripe cannot reach your localhost server. The Stripe CLI solves this by creating a tunnel that forwards webhook events to your local machine.

Install the Stripe CLI

```
# macOS
brew install stripe/stripe-cli/stripe

# Windows (via scoop)
scoop install stripe

# Linux
# Download from https://github.com/stripe/stripe-cli/releases
```

Authenticate and Start Forwarding

```
# Login with your Stripe account
stripe login

# Forward webhook events to your local Next.js app
stripe listen --forward-to localhost:3000/api/stripe/webhook

# The CLI will output a webhook signing secret for local use:
# > Ready! Your webhook signing secret is whsec_test_xxxx
```

```
# Add this to your .env.local as STRIPE_WEBHOOK_SECRET
```

Trigger Test Events Manually

```
# Simulate a successful checkout
stripe trigger checkout.session.completed

# Simulate a successful subscription renewal
stripe trigger invoice.paid

# Simulate a failed payment
stripe trigger invoice.payment_failed

# Simulate a subscription cancellation
stripe trigger customer.subscription.deleted
```

10.3 Testing Checklist

Before going live, verify every scenario in the table below works end-to-end in Test Mode:

Scenario	Expected Result
Monthly checkout completes	isPro = true, proType = 'monthly', stripeSubscriptionId populated
Lifetime checkout completes	isPro = true, proType = 'lifetime', proExpiresAt = null
Monthly renewal (invoice.paid)	isPro stays true, proExpiresAt updated to new period end
Payment failure (invoice.payment_failed)	isPro = false, user loses Pro access
Subscription cancellation	isPro = false, stripeSubscriptionId = null
Already lifetime user tries to upgrade	API returns 400 error, no duplicate charge
Unauthenticated user hits /api/stripe/checkout	Returns 401 Unauthorized
Pro feature accessed by free user	Returns 403 with upgrade prompt
Free user at design limit creates new design	Returns 403 with limit message
Customer Portal link generation	Redirects to Stripe-hosted portal page
Webhook signature tampered with	Returns 400, no database changes made

11. Error Handling

11.1 Stripe API Errors

Wrap all Stripe API calls in try-catch blocks. Stripe errors have a type field you can use to respond meaningfully.

```
try {
  const session = await stripe.checkout.sessions.create({ ... });
  return Response.json({ url: session.url });
} catch (err) {
  if (err.type === 'StripeCardError') {
    // Card was declined
    return Response.json({ error: err.message }, { status: 402 });
  } else if (err.type === 'StripeInvalidRequestError') {
    // Invalid parameters — likely a bug in your code
    console.error('Stripe invalid request:', err.message);
    return Response.json({ error: 'Configuration error' }, { status: 500 });
  } else {
    // Network error or Stripe outage
    console.error('Stripe error:', err);
    return Response.json({ error: 'Payment service unavailable' }, { status: 503
});
  }
}
```

11.2 Idempotent Webhook Handling

Stripe may deliver the same webhook event more than once. Always write your webhook handlers to be idempotent — running the same handler twice should produce the same result as running it once. The database update patterns shown in Section 7 are already idempotent since they use upsert-style updates.

Best Practice

Log all incoming webhook events to a separate database table with their event ID. Before processing, check if that event ID has already been handled. This prevents duplicate Pro grants or revocations on replay.

12. Going Live Checklist

When you are ready to accept real payments, complete every item in this checklist before switching to live mode.

12.1 Stripe Dashboard

- Complete Stripe account activation — business details, bank account, identity verification
- Create the same Product and Prices in Live Mode (they are separate from Test Mode)
- Register a new live webhook endpoint pointing to your production URL
- Copy the live webhook signing secret

12.2 Environment Variables

- Replace sk_test_ with sk_live_ in STRIPE_SECRET_KEY
- Replace pk_test_ with pk_live_ in NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY
- Replace whsec_test_ with the live webhook secret in STRIPE_WEBHOOK_SECRET
- Replace price_test_ IDs with the live Price IDs in both STRIPE_MONTHLY_PRICE_ID and STRIPE_LIFETIME_PRICE_ID

Never Mix Keys

Do not mix test and live keys. A test secret key with a live price ID will throw errors. A live secret key with test price IDs will fail silently. Double-check all four environment variables before deploying.

12.3 Final Verification

- Make a real $1.00 test purchase with your own card in live mode, then immediately refund it from the Stripe Dashboard
- Confirm the webhook fires and isPro is set correctly in your production database
- Confirm the Customer Portal link works and shows real billing history
- Test the cancellation flow and confirm Pro is revoked correctly
- Enable Stripe Radar (fraud detection) in Dashboard → Radar — it is free and catches fraudulent cards automatically

13. Common Mistakes to Avoid

Mistake	Why It Matters & What To Do Instead
Unlocking Pro on the success URL redirect	The success URL can be visited by anyone manually. Always grant Pro access via the webhook only.
Not verifying the webhook signature	Without signature verification, anyone can POST to your webhook and grant themselves Pro for free. Always call constructEvent with your signing secret.
Creating multiple Stripe Customers per user	Check for an existing stripeCustomerId before creating a new Customer. Duplicates cause billing confusion and cannot easily be merged.
Using test keys in production	Test mode transactions do not charge real cards. Verify your live keys are in place before launch.
Not handling invoice.paid for renewals	If you only handle checkout.session.completed, monthly subscribers lose Pro access after their first billing cycle because you never confirm subsequent renewals.
Storing raw card data	Never attempt to collect or store card numbers. Use Stripe's hosted Checkout or Elements exclusively. Handling raw card data requires PCI DSS Level 1 compliance.
Not testing payment failure paths	The happy path is easy. Always test invoice.payment_failed and customer.subscription.deleted so failed payments and cancellations are handled gracefully.
Forgetting idempotency on webhooks	Stripe can send the same event more than once. Build webhook handlers that produce the same outcome if run multiple times.

14. Stripe Fees & Revenue Impact

14.1 Stripe Transaction Fees

Stripe charges a fee per successful transaction. There are no monthly platform fees — you only pay when you earn.

Transaction Type	Fee
Standard card (Visa, Mastercard)	2.9% + $0.30 per transaction
International cards	+1.5% surcharge on top of standard rate
Currency conversion	+1.0% if currency conversion is required
Refunds	Processing fee not returned, but no additional charge
Disputes / chargebacks	$15 fee per dispute — reversed if you win

14.2 Net Revenue Per Plan

Scenario	Estimated Net Revenue
Monthly Pro at $7.00	$7.00 - ($0.20 + $0.30) = ~$6.50 net per month per subscriber
Lifetime at $35.00	$35.00 - ($1.02 + $0.30) = ~$33.68 net per sale
100 monthly subscribers	~$650 net MRR
200 lifetime sales at launch	~$6,736 net one-time revenue

14.3 Break-Even Analysis

With the Vercel + Neon + Resend infrastructure stack costing approximately $59/month at the early growth stage, PixelPush breaks even at 10 active Pro subscribers — achievable within the first week of a well-executed launch.

15. Glossary

Term	Definition
Customer	A Stripe object representing one of your users. Stores payment methods, subscriptions, and billing history. Identified by cus_xxx ID.
Product	What you are selling. In PixelPush this is 'PixelPush Pro'. Defined once and shared across pricing tiers.
Price	How you are selling a Product — amount, currency, and frequency. PixelPush has two: $7/month recurring and $35 one-time.
Checkout Session	A Stripe-hosted payment page session. Created server-side and presented to the user via URL redirect.
Webhook	An HTTP POST from Stripe to your server notifying you of a payment event. The authoritative source of truth for billing state.
Signing Secret	A secret key (whsec_xxx) used to verify that incoming webhook requests genuinely originated from Stripe.
Customer Portal	A Stripe-hosted page where subscribers manage their billing — update cards, view invoices, cancel subscriptions.
Test Mode	Stripe's sandbox environment. Transactions use test card numbers and never charge real money.
Live Mode	Stripe's production environment. Real card charges occur. Requires separate API keys from Test Mode.
PCI DSS	Payment Card Industry Data Security Standard. Compliance requirement for handling card data. Using Stripe Checkout keeps PixelPush out of PCI scope.
Idempotency	The property of an operation that produces the same result when run multiple times. Webhook handlers must be idempotent to handle duplicate event delivery safely.
MRR	Monthly Recurring Revenue. Total monthly income from all active Pro subscriptions.
Stripe Radar	Stripe's built-in fraud detection system. Free to use and blocks fraudulent cards automatically.

PixelPush — Stripe Integration Guide v1.0

Aepostrophee · Confidential · 2025